PROJECT REPORT

LAB211

Project Title: Car Showroom Management
Student: Nguyen Hoang Minh
Student ID: SE203563
Instructor: Ms. Le Vo Minh Thu

Contents

1. System Design Overview 1
   1.1. Use Case Diagram 1
   1.2. Class Diagram 1
2. Application of Computational Thinking (CT) 2
   2.1. Decomposition 2
   2.2. Pattern Recognition 3
   2.3. Abstraction 4
   2.4. Algorithm Design 4
3. Object-Oriented Programming (OOP) Implementation 6
   3.1. Data Models / Classes 6
   3.2. Encapsulation 6
   3.3. Abstraction 7
   3.4. Polymorphism 7
   3.5. Inheritance 8
4. Description of Core Modules and Functionalities 8
   4.1. Utility Module (Validation) 8
   4.2. Data Access Module (DAO – Read/Write Files) 11
   4.3. Business Logic Module (CRUD & Display) 15
5. Code Standards and Conclusion 20
   5.1. Code Quality Standards 20
   5.2. Conclusion 20

List of Diagram

Use case diagram 1
Class diagram 2

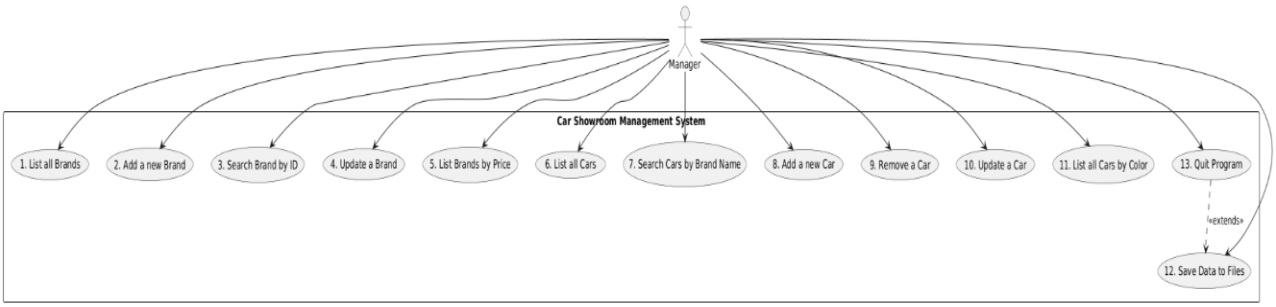
1. System Design Overview

Based on the problem requirements analysis, I designed the system using an object-oriented approach, focusing on the *Separation of Concerns* principle and applying core software design principles.

The following UML diagrams illustrate the structural and behavioral aspects of the system.

1.1. Use Case Diagram

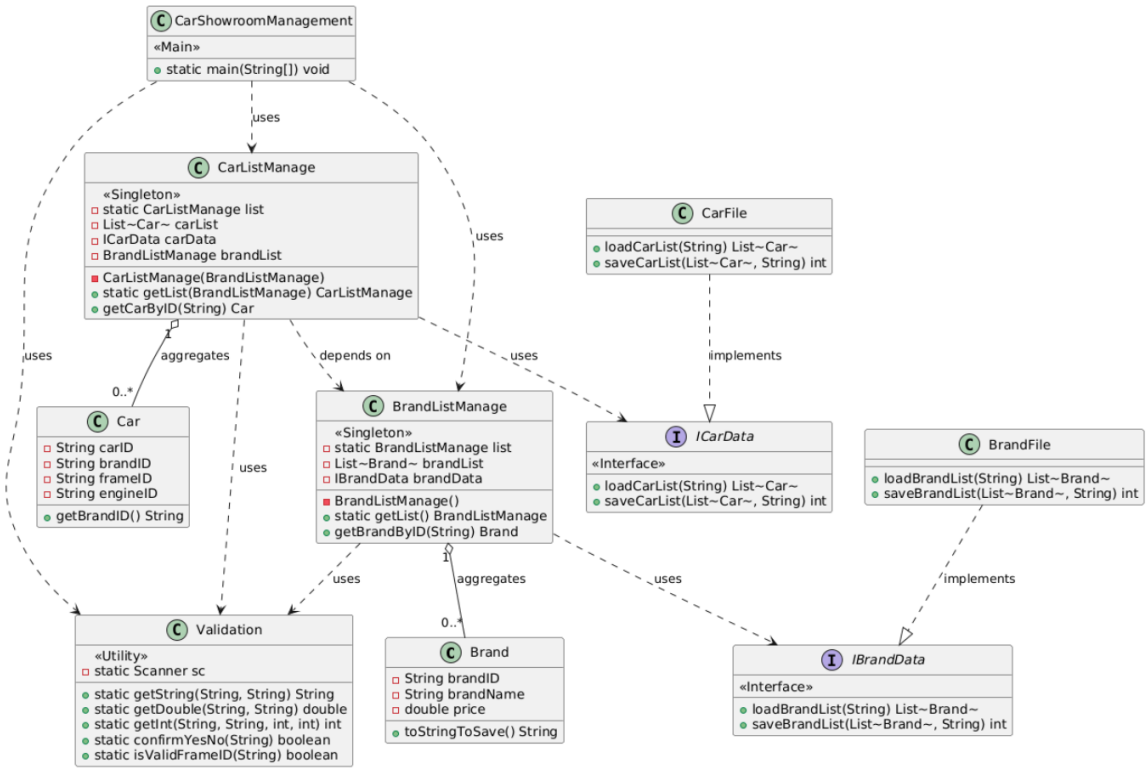
- **Description:** The *Use Case Diagram* illustrates the interactions between the user (Actor — referred to as the **Manager**) and the main functional components of the system.
- The system comprises **13 core functions**, each of which is directly invoked by the Manager.
- The diagram also demonstrates an **<<extends>> relationship**, in which the *Quit Program* function extends (or calls) the *Save Data* function whenever any modifications occur (dataChanged == true).
- **Use Case Diagram:**



1.2. Class Diagram

- **Description:** The *Class Diagram* provides a detailed view of the system's static structure, including its classes, interfaces, and the relationships among them.
- **Main Components:**
  - **Model** (Brand, Car): Classes responsible for storing data.
  - **Utility** (Validation): A helper class (<<Utility>>) containing static methods for input validation.
  - **DAO Pattern** (IBrandData, ICarData, BrandFile, CarFile): Interfaces I...Data abstract the file-access logic, while the corresponding ...File classes implement this logic.
  - **Singleton Pattern** (BrandListManage, CarListManage): Business management classes designed as singletons (<<Singleton>>) to ensure that only one instance of each list exists at any given time.

- **Key Relationships:**
  - CarListManage **depends on** BrandListManage (for brand lookup).
  - Classes ending with ...Manage **use** (uses) the I...Data interfaces (representing abstraction).
- **Class Diagram:**



2. Application of Computational Thinking (CT)

Computational Thinking (CT) is a problem-solving process. In this project, the four pillars of CT were applied to decompose complex business requirements into a well-structured software system.

2.1. Decomposition

Definition: Decomposition is the process of breaking down the complex problem *"Showroom Management"* into smaller, more manageable subproblems.

Application: Instead of solving all 13 core functions in a single file, the system was decomposed into **five separate functional modules**, each with a single, clearly defined responsibility.

a. Model Module (Data):

- Includes Brand.java and Car.java.

- These classes define the data structure "blueprints," containing attributes and their getter/setter methods.

b. DAO Module (Data Access):

- Includes interfaces IBrandData, ICarData, and their implementations BrandFile, CarFile.
- This module handles reading and writing data to text files.

c. Service Module (Business Logic):

- Includes BrandListManage.java and CarListManage.java.
- Responsible for processing all business logic (add, edit, delete, search, etc.).

d. Utility Module (Helper):

- Includes Validation.java.
- This module performs validation on all user input.

e. Controller Module (Program Control):

- Includes CarShowroomManagement.java.
- Contains the main() method and manages menu display and program flow.

2.2. Pattern Recognition

Definition: After decomposition, recurring "patterns" are identified across subproblems to design reusable solutions and avoid duplicated code.

Application:

- **Pattern 1 – CRUD & IO:**

  Both Brand and Car share common CRUD (Create, Read, Update, Delete) logic and file read/write (IO) operations.

  → **Solution:** Apply the *DAO Pattern* (Data Access Object).

  Create common interfaces IBrandData and ICarData for IO operations, separating business logic from data access.

- **Pattern 2 – Singleton List:**

  The system requires only one list of Brands and one list of Cars throughout the application.

  All operations (Add, Edit, Delete, Update) must act on a single shared instance.

  → **Solution:** Apply the *Singleton Pattern* for BrandListManage and CarListManage.

- **Pattern 3 – Repeated Input Validation:**

Most functions (Add, Update, Search, Filter) require repeated user input and validation (non-empty, format, range checks).

→ **Solution:** Apply the *Utility Pattern* (Section 9) with Validation.java, defining reusable static methods (e.g., getString, getInt) to centralize validation logic.

2.3. Abstraction

Definition: Abstraction hides implementation complexity while retaining only essential properties and behaviors.

Application:

- **Data Abstraction (Section 5):**

  Classes Brand and Car store only essential attributes (ID, name, price...).

  Users do not need to know how this data is stored or processed internally.

- **Behavioral Abstraction (Section 8 – Interfaces):**

  A clear example:

  The business class BrandListManage does not need to know how BrandFile reads text files (e.g., using BufferedReader, split(","), or split(" ")).

  It only interacts through the interface "contract" IBrandData by invoking brandData.loadBrandList().

  The file-handling complexity is fully hidden within the BrandFile implementation.

2.4. Algorithm Design

Definition: Algorithm Design defines step-by-step workflows to clearly solve each functionality before coding.

Application (Example 1): Add New Car (Function 8)

Algorithm Steps:

1. **Start**
2. Request input carID using Validation.getString.
3. **Check Logic:**

   Call getCarByID(carID) to verify whether the ID exists.

   → If it does, repeat Step 2.
4. Display Brand list using brandList.listAllBrands().
5. Request input brandID.

6. **Check Logic:**

   Use brandList.getBrandByID(brandID) to verify whether Brand ID exists.

   → If not, repeat Step 5.
7. Request input color.
8. Request input frameID.
9. **Format Validation:**

   Use Validation.isValidFrameID() to check frame format Fxxxxx.
10. **Logic Validation:**

    Use isFrameIDUnique() to ensure uniqueness.

    → If duplicated, repeat Step 8.
11. Request input engineID (similar validation with isValidEngineID, isEngineIDUnique).
12. Create new Car(...) object and add to carList.
13. Set Validation.dataChanged = true.
14. **End.**

Algorithm Example 2: Sort and Display Cars (Function 6)

1. Create a copy of the list:

   List<Car> sortedList = new ArrayList<>(this.carList);
2. Define the comparison strategy using Comparator<Car>.
3. In the compare(Car c1, Car c2) method:
   - Get Brand b1, Brand b2 via c1.getBrandID(), c2.getBrandID().
   - Compare brand names name1, name2.
   - If names differ → return nameCompare.
   - If names match → compare price1, price2 using Double.compare(price1, price2).
   - Return sorting order (ascending).
4. Sort using Collections.sort(sortedList, carComparator);
5. Display the sorted list (sortedList, not carList).

3. Object-Oriented Programming (OOP) Implementation

To solve the problem using the object-oriented approach, the system was developed based on fundamental OOP principles.

3.1. Data Models / Classes

Definition: These are *entity/model* classes representing real-world objects. They contain data attributes and basic methods to access or modify that data.

Application: The system defines two main model classes:

1. **Brand.java** – Represents a brand entity.

   This class contains the fields specified in the project requirements (from brands.txt), including: brandID, brandName, soundBrand, and price.

2. **Car.java** – Represents a car entity.

   This class includes fields (from cars.txt) such as: carID, brandID (linked to Brand), color, frameID, and engineID.

3.2. Encapsulation

Definition: Encapsulation (data hiding) is the principle of restricting direct access to object data. Object attributes must be protected to prevent unauthorized or unsafe external access.

Application:

- All fields in both model classes (Brand and Car) are declared as private.

  Example (in Brand.java):

  ```
  public class Brand {
    private String brandID;
    private String brandName;
    // ...
  }
  ```

- To access or modify these private values, external classes (e.g., BrandListManage) must use *public* methods — specifically:
  - **Getters** (e.g., getBrandName()), and
  - **Setters** (e.g., setBrandName(...)).
- **This** ensures data integrity and allows validation or control before updating values, thereby improving the reliability and consistency of the system's data management.

3.3. Abstraction

Definition: Abstraction hides the complexity of implementation details while exposing only the necessary functionalities to the outside. The interface is one of the most powerful mechanisms to achieve abstraction in object-oriented design.

Application (DAO Pattern):

- Multiple **interfaces (contracts)** such as IBrandData, ICarData were defined.
- These interfaces specify the mandatory behaviors (methods) such as loadBrandList() and saveBrandList(), without describing *how* the operations are implemented.
- Business logic classes (e.g., BrandListManage) only *know about* and *work with* the IBrandData interface.

  The underlying file-handling complexity (using BufferedReader, PrintWriter, split(","), etc.) is completely hidden within the implementing classes (BrandFile, CarFile).

This design ensures that the business layer is entirely decoupled from the data access implementation.

3.4. Polymorphism

Definition: Polymorphism means "one object, many forms.". It allows a reference variable of an interface (or a parent class) type to point to objects of different implementing (or subclass) types.

Application (DAO Pattern): Polymorphism is clearly demonstrated in the management classes.

Example (in BrandListManage.java):

```
public class BrandListManage {
    // 1. Declare interface variable (abstract type)
    private IBrandData brandData;

    private BrandListManage() {
        // 2. Instantiate with a concrete class
        this.brandData = new BrandFile();
    }
}
```

- Here, the variable brandData is of type IBrandData (abstract).

- At runtime, it is *polymorphically bound* to a specific object of type BrandFile.
- **Advantage:**
  If, in the future, data needs to be stored in a database instead of text files, only the implementation class needs to change (e.g.,

  BrandDatabaseFile implements IBrandData).

  Then, simply update the instantiation:

  this.brandData = new BrandDatabaseFile();

All business logic in BrandListManage remains unchanged — demonstrating the flexibility and maintainability provided by polymorphism.

3.5. Inheritance

Definition: Inheritance allows a class (subclass) to acquire the attributes and methods of another class (superclass).

Application:

- In this project, inheritance (via the extends keyword) is *not* the primary design approach.
- Instead, the project favors the use of **Interfaces** (as shown in the DAO Pattern) and **Composition** (the "has-a" relationship) over direct inheritance.

  Example (Composition):

  The class CarListManage **has-a** member instance of BrandListManage:

  private BrandListManage brandList;

  This relationship allows CarListManage to look up brand information. Such a design is more flexible than classical inheritance because CarListManage is *not a subtype* of BrandListManage (i.e., it does not represent an "is-a" relationship).

4. Description of Core Modules and Functionalities

This section provides a detailed explanation of how the main modules operate to execute the 13 core functions of the system.

4.1. Utility Module (Validation)

Class: Validation.java

Design: This module follows the **Utility Pattern**. All its methods are declared as static, and the class has a private constructor (added to prevent object instantiation). It maintains a single static final Scanner instance to avoid NoSuchElementException caused by multiple System.in inputs.

Functionality: Provides utility methods to retrieve and validate user input, including:

- getString(message, error): Retrieves a non-empty string.
- getInt(message, error, min, max): Retrieves an integer within the range [min, max].
- getDouble(message, error): Retrieves a positive double value (> 0).
- updateString(message, oldValue) / updateDouble(...): Handles update logic (pressing Enter keeps the old value).
- isValidFrameID(id) / isValidEngineID(id): Validates string format using RegEx — "F\\d{5}" and "E\\d{5}".
- confirmYesNo(message): Prompts user confirmation (Y/N).

```java
public class Validation {
    private static final String FRAME_ID_PATTERN = "F\\d{5}";
    private static final String ENGINE_ID_PATTERN = "E\\d{5}";
    public static boolean dataChanged = false;
    private static final Scanner sc = new Scanner(System.in);

    // Frame ID must follow the format: "F00000"
    public static boolean isValidFrameID(String frameID) {
        return frameID != null && frameID.matches(FRAME_ID_PATTERN);
    }

    // Engine ID must follow the format: "E00000"
    public static boolean isValidEngineID(String engineID) {
        return engineID != null && engineID.matches(ENGINE_ID_PATTERN);
    }

    // Brand ID in Car must not be null or empty
    public static boolean isValidCarBrandID(String brandID) {
        return brandID != null && !brandID.trim().isEmpty();
    }

    public static String getString(String welcome, String error) {
        String result = null;
        do {
            System.out.print(welcome);
            result = sc.nextLine().trim();
            if (result.isEmpty()) {
                System.out.println(error);
            }
        } while (result.isEmpty());
        return result;
    }
```

```java
    public static String updateString(String welcome, String oldValue) {
        String result = oldValue;
        System.out.print(welcome);
        result = sc.nextLine();
        if (result.isEmpty()) {
            result = oldValue;
        }
        return result;
    }

    public static double getDouble(String welcome, String error) {
        double result = 0;
        boolean contionous = true;
        do {
            System.out.print(welcome);
            try {
                result = Double.parseDouble(sc.nextLine().trim());
                if (result <= 0) {
                    System.out.println(error);
                } else {
                    contionous = false;
                }
            } catch (Exception e) {
                System.out.println(error);
            }
        } while (contionous);
        return result;
    }
```

```java
    public static double updateDouble(String welcome, String error, double oldValue) {
        double result = oldValue;
        boolean contionous = true;
        do {
            System.out.print(welcome);
            try {
                String input = sc.nextLine().trim();
                if (input.isEmpty()) {
                    result = oldValue;
                    contionous = false;
                } else {
                    result = Double.parseDouble(input);
                    if (result <= 0) {
                        System.out.println(error);
                    } else {
                        contionous = false;
                    }
                }
            } catch (Exception e) {
                System.out.println(error);
            }
        } while (contionous);
        return result;
    }
```

```java
public static int getInt(String welcome, String error, int min, int max) {
    int result = 0;
    boolean continous = true;
    do {
        System.out.print(welcome);
        try {
            result = Integer.parseInt(sc.nextLine().trim());
            if (result > max || result < min) {
                System.out.println("Choice must be between " + min + " and " + max + ".");
            } else {
                continous = false;
            }
        } catch (Exception e) {
            System.out.println(error);
        }
    } while (continous);
    return result;
}
```

```java
public static String getUpdateStringByRegex(String message, String oldValue, String error_message, String regex) {
    while (true) {
        System.out.print(message);
        String input = sc.nextLine().trim();

        if (input.isEmpty()) {
            return oldValue;
        }

        if (input.matches(regex)) {
            return input;
        } else {
            System.out.println(error_message);
        }
    }
}

public static boolean confirmYesNo(String welcome) {
    String input = getString(welcome, "Please enter Y or N.").trim();
    return input.equalsIgnoreCase("Y");
}

public static boolean isDataChanged() {
    return dataChanged;
}
}
```

4.2. Data Access Module (DAO – Read/Write Files)

Classes: BrandFile.java, CarFile.java (implementing IBrandData and ICarData interfaces)

Purpose: These classes handle file I/O operations — reading from and writing to text files.

Algorithm for Data Loading *(Example: loadBrandList())*

1. Initialize an empty ArrayList<Brand>.
2. Use FileReader and BufferedReader to read brands.txt. (Check file existence before reading.)
3. Read each line using r.readLine() within a while (r.ready()) loop.
4. Split the line by "," into three parts.

5. Ensure arr.length == 3 to validate input format.
6. Further split arr[2] by ": " to separate soundBrand and price.
7. Validate brandAndPrice.length == 2 to prevent Exception.
8. Clean and convert price string using replace("B", "") and Double.parseDouble().
9. Create a new Brand object and add it to the list.
10. Use a finally block to ensure that FileReader and BufferedReader are properly closed (f.close(), r.close()), even if an exception occurs.

Algorithm for Data Saving *(Example: saveBrandList())*

1. Use PrintWriter to open the brands.txt file.
2. Iterate through each Brand object in the List<Brand>.
3. Call brand.toStringToSave() to retrieve formatted data (ID, Name, Sound, PriceB) and write using f.println().
4. Call f.flush() to ensure all data is written to the file.
5. Use a finally block to close the PrintWriter (f.close()).

```
public interface IBrandData {
    List<Brand>  loadBrandList(String path);
    int saveBrandList(List<Brand> brandList, String path);
}
public interface ICarData {
    List<Car>  loadCarList(String path);
    int saveCarList(List<Car> carList, String path);
}
```

```java
public class CarFile implements ICarData{
    @Override
    public List<Car> loadCarList(String path) {
        List<Car> list = new ArrayList<>();
        FileReader f = null;
        BufferedReader r = null;
        try {
            File fileCheck = new File(path);
            if(fileCheck.exists() && fileCheck.isFile()){
                f = new FileReader(path);
                r = new BufferedReader(f);
                while (r.ready()){
                    String s = r.readLine();
                    String[] arr = s.split(",");
                    if (arr.length == 5){
                        String carID = arr[0].trim();
                        String brandID = arr[1].trim();
                        String color = arr[2].trim();
                        String frameID = arr[3].trim();
                        String engineID = arr[4].trim();
                        Car c = new Car(carID, brandID, color, frameID, engineID);
                        list.add(c);
                    }
                }
            }
        } catch (Exception e) {
            System.out.println("Error Loading file!!!!!!");
        } finally {
            try {
                if(f!=null) f.close();
                if(r!=null) r.close();
            } catch (Exception e) {
            }
        }
        return list;
    }
```

```java
    @Override
    public int saveCarList(List<Car> carList, String path) {
        int check = 0;
        PrintWriter f = null;
        try {
            f = new PrintWriter(path);
            for (Car car : carList) {
                String c = car.getCarID() + "," + car.getBrandID() + "," + car.getColor() + "," + car.getFrameID() + "," + car.getEngineID() + "\n";
                f.print(c);
                f.flush();
            }
        } catch (Exception e) {
            check = -1;
        } finally {
            try {
                if(f!=null) f.close();
            } catch (Exception e) {
                check = -2;
            }
        }
        return check;
    }
}
```

```java
public class BrandFile implements IBrandData{
    @Override
    public List<Brand> loadBrandList(String path) {
        List<Brand> list = new ArrayList<>();
        FileReader f = null;
        BufferedReader r = null;
        try {
            File fileCheck = new File(path);
            if(fileCheck.exists() && fileCheck.isFile()){
                f = new FileReader(path);
                r = new BufferedReader(f);
                while (r.ready()){
                    String s = r.readLine();
                    String[] arr = s.split(",");
                    if (arr.length == 3){
                        String brandID = arr[0].trim();
                        String brandName = arr[1].trim();
                        String[] brandAndPrice = arr[2].split(": ");
                        if (brandAndPrice.length == 2) { //phai co if de tranh crash chuong trinh
                            String soundBrand = brandAndPrice[0].trim();
                            double price = Double.parseDouble(brandAndPrice[1].trim().replace("B", ""));
                            Brand b = new Brand(brandID, brandName, soundBrand, price);
                            list.add(b);
                        }
                    }
                }
            }
        } catch (Exception e) {
            System.out.println("Error Loading file!!!!!!");
        } finally {
            try {
                if(f!=null) f.close();
                if(r!=null) r.close();
            } catch (Exception e) {
            }
        }
        return list;
    }
```

```java
    @Override
    public int saveBrandList(List<Brand> brandList, String path) {
        int check = 0;
        PrintWriter f = null;
        try {
            f = new PrintWriter(path);
            for (Brand brand : brandList) {
                f.println(brand.toStringToSave());
                f.flush();
            }
        } catch (Exception e) {
            check = -1;
        } finally {
            try {
                if(f!=null) f.close();
            } catch (Exception e) {
                check = -2;
            }
        }
        return check;
    }
}
```

4.3. Business Logic Module (CRUD & Display)

Classes: BrandListManage.java, CarListManage.java

Purpose: This module represents the *"brain"* of the system — it coordinates between the DAO and Validation modules to execute all core operations.

Display (Section 10):

- Uses System.out.printf() and String.format() for column alignment (e.g., "%-10s %-40s").
- Uses visual separators such as new String(new char[90]).replace("\0", "-") to improve readability.

```java
private void headerForBrandList() {
    System.out.println("\n===================================== BRAND LIST =====================================");
    System.out.printf(String.format("%-10s | %-40s | %-20s | %s\n", "ID", "Brand's name", "Sound brand", "Price"));
    System.out.println(new String(new char[90]).replace("\0", "-"));
}

private void headerForSearchResult() {
    System.out.println("\n==================================== SEARCH RESULTS ====================================");
    System.out.printf(String.format("%-10s | %-40s | %-20s | %s\n", "ID", "Brand's name", "Sound brand", "Price"));
    System.out.println(new String(new char[90]).replace("\0", "-"));

}

private void printBrandListDetails(List<Brand> list) {
    for (Brand brand: list) {
            System.out.println(brand.toString());
    }
}

public void listAllBrands() {
    if (brandList.isEmpty()) {
        System.out.println("Brand List is empty!!!!\n");
    } else {
        headerForBrandList();
        printBrandListDetails(brandList);
        System.out.println(new String(new char[90]).replace("\0", "-") +"\n");
    }
}
```

```java
public void listBrandByPrice() {
    double inputPrice = Validation.getDouble("Enter a maximum price (Billion): ", "This field cannot be empty!!!!");
    List<Brand> result = new ArrayList<>();
    for (Brand brand : brandList) {
        if (brand.getPrice() <= inputPrice) {
            result.add(brand);
        }
    }
    if (result.isEmpty()) {
        System.out.println("No brand has the price under " + inputPrice + "B.");
    } else {
        headerForSearchResult();
        System.out.println("                                    (Brand has the price under " + inputPrice + "B)");
        printBrandListDetails(result);
        System.out.println(new String(new char[90]).replace("\0", "-") + "\n");
    }
}
```

```java
private void headerForCarList() {
    System.out.println("\n==================================== CAR LIST ====================================");
    System.out.printf(String.format("%-10s | %-30s | %-10s | %-10s | %-10s\n", "Car ID", "Brand name", "Color", "Frame ID", "Engine ID"));
    System.out.println(new String(new char[84]).replace("\0", "-"));
}

private void headerForSearchResult() {
    System.out.println("\n================================= SEARCH RESULTS =================================");
    System.out.printf(String.format("%-10s | %-30s | %-10s | %-10s | %-10s\n", "Car ID", "Brand name", "Color", "Frame ID", "Engine ID"));
    System.out.println(new String(new char[84]).replace("\0", "-"));
}

private void printACarDetails(Car car) {
    Brand brand = brandList.getBrandByID(car.getBrandID());
    String brandName;
    if (brand != null) {
        brandName = brand.getBrandName();
    } else {
        brandName = "N/A";
    }
    System.out.printf(String.format("%-10s | %-30s | %-10s | %-10s | %-10s\n", car.getCarID(), brandName, car.getColor(), car.getFrameID(), car.getEngineID()));
}
```

```java
public void listCarByColor() {
    String color = Validation.getString("Enter car's color that you want to find: ", "This field cannot be empty!!!!");
    List<Car> result = new ArrayList<>();
    for (Car car : carList) {
        if (car.getColor().equalsIgnoreCase(color)) {
            result.add(car);
        }
    }
    if (result.isEmpty()) {
        System.out.println("No cars found this color " + color);
    } else {
        headerForSearchResult();
        for (Car car : result) {
            printACarDetails(car);
        }
        System.out.println(new String(new char[84]).replace("\0", "-") + "\n");
    }
}
```

```java
public void listAllCarsSorted() {
    if (carList.isEmpty()) {
        System.out.println("Car list is empty!!!!");
    } else {
        List<Car> sortedList = new ArrayList<>(this.carList);
        Comparator<Car> carCompare = new Comparator<Car>() {
            @Override
            public int compare(Car c1, Car c2) {
                Brand b1 = brandList.getBrandByID(c1.getBrandID());
                Brand b2 = brandList.getBrandByID(c2.getBrandID());
                String name1;
                String name2;
                if (b1 != null) {
                    name1 = b1.getBrandName();
                } else {
                    name1 = "";
                }
                if (b2 != null) {
                    name2 = b2.getBrandName();
                } else {
                    name2 = "";
                }
                int nameCompare = name1.compareTo(name2);
                if (nameCompare == 0) {
                    double price1;
                    double price2;
                    if (b1 != null) {
                        price1 = b1.getPrice();
                    } else {
                        price1 = 0;
                    }
                    if (b2 != null) {
                        price2 = b2.getPrice();
                    } else {
                        price2 = 0;
                    }
                    return Double.compare(price2, price1);
                }
                return nameCompare;
            }

        };
        Collections.sort(sortedList, carCompare);
        headerForCarList();
        for (Car car : sortedList) {
            printACarDetails(car);
        }
        System.out.println(new String(new char[84]).replace("\0", "-") + "\n");
    }
}
```

Add (Section 11):

- Methods addNewBrand() and addNewCar() call Validation methods (e.g., getString, getDouble) to collect input.
- Performs logical validation such as checking for existing IDs (getBrandByID(brandID) == null).
- Upon validation success, creates a new Brand(...) or Car(...) object and appends it to the ArrayList.
- Sets Validation.dataChanged = true to notify the system that data must be saved before exiting.

```java
public void addNewCar() {
    System.out.println("========================= ADD A NEW CAR =========================");
    String carID;
    while (true) {
        carID = Validation.getString("Enter car's ID: ", "This field cannot be empty!!!!");
        if (getCarByID(carID) == null) {
            break;
        } else {
            System.out.println("The ID '" + carID + "' has been existed!!!!");
        }
    }
    System.out.println("The list below is the list of brand, you must choose 1 from the list!!!!");
    brandList.listAllBrands();
    String brandID;
    while (true) {
        brandID = Validation.getString("Enter brand's ID that you choose from the brand list: ", "This field cannot be empty!!!!");
        if (brandList.getBrandByID(brandID) != null) {
            break;
        } else {
            System.out.println("The brand ID '" + brandID + "' is not exist!!!!");
        }
    }
    String color = Validation.getString("Enter car's color: ", "This field cannot be empty!!!!");
    String frameID;
    while (true) {
        frameID = Validation.getString("Enter car's frame ID (must follow the format 'Fxxxxx'): ", "This field cannot be empty!!!!");
        if (Validation.isValidFrameID(frameID)) {
            if (isFrameIDUnique(frameID)) {
                break;
            } else {
                System.out.println("The frame ID '" + frameID + "' has been existed!!!!");
            }
        } else {
            System.out.println("Wrong format!!!!");
        }
    }
    String engineID;
    while (true) {
        engineID = Validation.getString("Enter car's engine ID (must follow the format 'Exxxxx'): ", "This field cannot be empty!!!!");
        if (Validation.isValidEngineID(engineID)) {
            if (isEngineIDUnique(engineID)) {
                break;
            } else {
                System.out.println("The engine ID '" + engineID + "' has been existed!!!!");
            }
        } else {
            System.out.println("Wrong format!!!!");
        }
    }
    Car car = new Car(carID, brandID, color, frameID, engineID);
    this.carList.add(car);
    System.out.println("Successfully added to list for car ID: " + carID);
    Validation.dataChanged = true;
}
```

```java
public void addNewBrand() {
    System.out.println("========================= ADD A NEW BRAND =========================");
    String brandID;
    while (true) {
        brandID = Validation.getString("Enter brand's ID: ", "This field cannot be empty!!!!");
        if (getBrandByID(brandID) == null) {
            break;
        } else {
            System.out.println("The ID '" + brandID + "' has been existed!!!!");
        }
    }
    String brandName = Validation.getString("Enter brand's name: ", "This field cannot be empty!!!!");
    String soundBrand = Validation.getString("Enter sound's brand: ", "This field cannot be empty!!!!");
    double price = Validation.getDouble("Enter brand's price (Billion): ", "This field cannot be empty!!!!");
    Brand newBrand = new Brand(brandID, brandName, soundBrand, price);
    this.brandList.add(newBrand);
    System.out.println("Successfully added to list for brand ID: " + brandID +"\n");
    Validation.dataChanged = true;
}
```

Search (Section 12):

- Methods getBrandByID(id) and getCarByID(id) iterate through the ArrayList using a for-each loop.
- Uses .equalsIgnoreCase(id.trim()) to perform case-insensitive matching and ignore extra spaces.
- searchCarByBrandName() supports partial name matching using:

  brand.getBrandName().toLowerCase().contains(partialName.toLowerCase());

```java
public void searchBrandByID() {
    String brandID = Validation.getString("Enter brand's ID: ", "This field cannot be empty!!!!");
    Brand brand = getBrandByID(brandID);
    if (brand == null) {
        System.out.println("This brand does not exist!!!!\n");
    } else {
        headerForSearchResult();
        System.out.println(brand.toString());
        System.out.println(new String(new char[90]).replace("\0", "-") + "\n");
    }
}
```

```java
public void searchCarByBrandName() {
    String partialName = Validation.getString("Enter a partial brand's name that you want to find: ", "This field cannot be empty!!!!");
    List<Car> result = new ArrayList<>();
    for (Car car : carList) {
        Brand brand = brandList.getBrandByID(car.getBrandID());
        if (brand != null && brand.getBrandName().toLowerCase().contains(partialName)) {
            result.add(car);
        }
    }
    if (result.isEmpty()) {
        System.out.println("No car found!!!!\n");
    } else {
        headerForSearchResult();
        for (Car car : result) {
            printACarDetails(car);
        }
        System.out.println(new String(new char[84]).replace("\0", "-") + "\n");
    }
}
```

Update (Section 13):

- Locates the target object using get...ByID().
- Calls Validation.updateString() or Validation.updateDouble() to handle input. (If the user presses Enter, the method retains oldValue.)
- For frameID and engineID (within updateCar()), the module ensures validity and uniqueness using loops with Validation.isValid... and isUnique checks.
- Sets Validation.dataChanged = true upon successful modification.

```java
public void updateBrand() {
    String brandID = Validation.getString("Enter brand's ID to update: ", "This field cannot be empty!!!!");
    Brand brand = getBrandByID(brandID);
    if (brand == null) {
        System.out.println("This brand does not exist!!!!\n");
    } else {
        System.out.println("===== UPDATE INFORMATION FOR BRAND " + brandID + " =====");
        System.out.println("(If you don't want to update a field, just press enter on that field.)");
        String newName = Validation.updateString("Enter new brand's name: ", brand.getBrandName());
        String newSoundBrand = Validation.updateString("Enter new sound's brand: ", brand.getSoundBrand());
        double newPrice = Validation.updateDouble("Enter new brand's price (Billion): ", "Invalid Price!!!", brand.getPrice());
        brand.setBrandName(newName);
        brand.setSoundBrand(newSoundBrand);
        brand.setPrice(newPrice);
        System.out.println(new String(new char[60]).replace("\0", "="));
        System.out.println("Successfully updated for brand ID: " + brandID + "\n");
    }
    Validation.dataChanged = true;
}
```

```java
public void updateCar() {
    String carID = Validation.getString("Enter car's ID to update: ", "This field cannot be empty!!!!");
    Car car = getCarByID(carID);
    if (car == null) {
        System.out.println("The car ID '" + carID + "' is not exist!!!!");
    } else {
        System.out.println("===== UPDATE INFORMATION FOR CAR " + carID + " =====");
        System.out.println("(If you don't want to update a field, just press enter on that field.)");
        String newColor = Validation.updateString("Enter new car's name: ", car.getColor());
        car.setColor(newColor);
        String frameID;
        while (true) {
            frameID = Validation.updateString("Enter new car's frame ID (must follow the format 'Fxxxxx'): ", car.getFrameID());
            if (frameID.equals(car.getFrameID())) {
                break;
            }
            if (Validation.isValidFrameID(frameID)) {
                if (isFrameIDUnique(frameID)) {
                    car.setFrameID(frameID);
                    break;
                } else {
                    System.out.println("The frame ID '" + frameID + "' has been existed!!!!");
                }
            } else {
                System.out.println("Wrong format!!!!");
            }
        }
        String engineID;
        while (true) {
            engineID = Validation.updateString("Enter new car's engine ID (must follow the format 'Exxxxx'): ", car.getEngineID());
            if (engineID.equals(car.getEngineID())) {
                break;
            }
            if (Validation.isValidEngineID(engineID)) {
                if (isEngineIDUnique(engineID)) {
                    car.setEngineID(engineID);
                    break;
                } else {
                    System.out.println("The engine ID '" + engineID + "' has been existed!!!!");
                }
            } else {
                System.out.println("Wrong format!!!!");
            }
        }
    }
    System.out.println(new String(new char[60]).replace("\0", "="));
    System.out.println("Successfully updated for brand ID: " + carID +"\n");
    Validation.dataChanged = true;
}
```

Delete (Section 14):

- Requests carID input from the user.
- Retrieves the corresponding Car object using getCarByID(carID).
- If found, removes it from the ArrayList using:

  this.carList.remove(car);

- Sets Validation.dataChanged = true to indicate that changes need to be saved.

```java
public void removeCar() {
    String carID = Validation.getString("Enter car's ID to remove: ", "This field cannot be empty!!!!");
    Car car = getCarByID(carID);
    if (car == null) {
        System.out.println("The car ID '" + carID + "' is not exist!!!!");
    } else {
        this.carList.remove(car);
        System.out.println("Successfully removed for car ID: " + carID + "\n");
    }
    Validation.dataChanged = true;
}
```

5. Code Standards and Conclusion

5.1. Code Quality Standards

Throughout the development process, all code quality standards were strictly followed to ensure the project remains readable, maintainable, and consistent.

a. Naming Conventions

All code follows Java naming standards:

- **PascalCase** (Capitalized first letter) for **Class** and **Interface** names: e.g., BrandListManage, ICarData, Brand, Car.
- **camelCase** (Lowercase first letter) for **method** and **variable** names: e.g., brandList, carData, getBrandByID(), addNewBrand().

b. Packaging

All .java files are grouped under the same package: carshowroommanagement.

c. Clean Code Principles

- **Single Responsibility Principle:** Each class is designed with one clear responsibility. For example:
  - Validation handles only user input validation.
  - BrandFile manages Brand file I/O operations.
  - BrandListManage handles business logic exclusively.
- **Meaningful Naming:** Variable and method names are descriptive and self-explanatory, such as: isFrameIDUnique(), listAllCarsSorted().
- **Comments:** Complex logic and important code segments are annotated for clarity, for example:
  - Comments explaining array validation: brandAndPrice.length == 2 in BrandFile.java.
  - Comments indicating “dependencies” in CarListManage.java.
- **Avoid Magic Numbers:** All numeric constants (e.g., FRAME_ID_PATTERN) are clearly named and defined to improve readability.

5.2. Conclusion

a. Results Achieved

The project successfully fulfills **100% of the functional requirements**, forming a complete Java Console application capable of executing all **13 main features** (from F1 to F9) effectively and accurately.

b. Lessons Learned

- Through this project, I gained a deeper understanding of the balance between **design and implementation**.
- Applying the four pillars of **Computational Thinking (CT)** — *Decomposition, Pattern Recognition, Abstraction,* and *Algorithm Design* — together with **Design Patterns** (*Singleton, DAO, Utility*) helped make the code modular, reusable, and scalable.

c. Report Summary

This report presents a clear, concise explanation of the project's logic, design structure, and implementation process. It includes both the application of **Computational Thinking principles** and the relevant UML diagrams (Use Case, Class, etc.) as supporting materials.

--- END ---